AYYILDIZ CONTEXT ENGINEERING FRAMEWORK

ACEF — Developer Guide

A stack-agnostic way to let an AI agent understand your project, route the work, and run only the steps that case needs. This document combines the Overview, the SDLC & frameworks detail, and the per-role guide.

AYYILDIZ CONTEXT ENGINEERING FRAMEWORK

ACEF Developer Guide

A practical guide for developers who will *use* ACEF on a real codebase — a stack-agnostic way to let an AI agent understand your project first, route the work correctly, and run only the steps that case needs. For the full process see SDLC & frameworks; for role-specific use see Roles.

🧭 What ACEF is

ACEF is not a code generator and not a second methodology. It is a thin layer that makes AI-assisted work fit *your* repo, doing three things:

Extract

Reads a **project adapter** from the repo — stack, commands, CI, tests, golden examples, risky areas — from evidence, not guesses.

Route

Classifies each request and runs only the SDLC steps that case needs. No ceremony.

Orchestrate

Drives existing tools (BMAD, Kiro, spec-kit, GSD, your own skills) through that adapter, behind one agent.

💬 How to use it

You don't pick routes, phases, or skills. You talk to the `acef` agent in plain language and it drives the rest:

YOU
I want to add a notifications screen to this app.

ACEF
Brownfield, small feature. I'll copy the screen pattern your repo already uses, implement, then review. Which area is the entry point — anything special?

YOU
Write tests for this service — there are none yet.

ACEF
No test pattern here yet. First I'll set up one example test in your stack and get your approval — that becomes the pattern. Then we expand. OK?

Replies are short by default (one step at a time). Ask for more depth anytime:

- `detail` → the route's purpose, inputs/outputs, why, risks
- `how` → which skills, adapter values, and files will run
- `teach` → explains the ACEF concept with an example

🚦 Routes — which one, when

The agent classifies your intent into one route and runs only its steps. You don't choose this — it's here so you understand what it's doing.

A · Small feature

localized, follows an existing pattern

Short brief → golden neighbor → implement → review.

B · Large feature

multi-module, new pattern, or touches data/auth/payment

Requirements → design → planning → build per task → review.

C · Bug fix

correct known broken behavior

Reproduce → root cause → minimal fix → regression test → review.

D · Test cases

produce test cases from the product

Flows → acceptance criteria → test cases.

E · Test automation

repo needs its first automation

Inspect tests → bootstrap one approved test → automate a flow.

F · Unit / integration

add coverage to a module

Find pattern (or bootstrap) → focused tests → run.

⬆️ **Escalation rule:** if the request is ambiguous, low-confidence, or touches a risk area (auth, payment, migration, contract, multi-repo), the agent moves *up* a route — never down — and asks at most a couple of yes/no questions.

🔍 Onboarding a repo

The first time ACEF touches a repo it extracts the **project adapter** (read-only). Everything else relies on it.

```
acef-adapter  →  {project}-adapter.md

Captures (each field cites a source path):
  identity / lifecycle · stack + package manager · layout · real commands
  (build / test / lint / typecheck) · test setup · CI/CD · golden neighbors · risk
  + freshness stamp (generated_at · commit · scope covered)
```

It does **not** install, write tests, or change code. It reads manifests, lock files, CI workflows, and the directory tree — no embeddings, no guessing. Re-run only when it goes stale (big refactor, new module, dependency or contract change), not for every task.

🛠️ Task how-to

Small feature

Adapter must be fresh → the agent picks a qualified golden neighbor → copies its pattern → review. No heavy spec/design.

Large feature

`acef-specify` imports BMAD/Kiro output if it exists (never regenerates), else lightweight requirements/design/tasks with traceable IDs → build per task → review.

Bug fix

Give expected vs actual + a repro. The agent finds root cause, makes the minimal fix matching local style, adds a regression test.

Tests (none yet)

`acef-test-bootstrap` detects the framework from evidence (or proposes the stack default and asks), writes **one** idiomatic test, runs it, and gets your approval — that becomes the golden test.

Release

`acef-release-adapter` builds a readiness checklist from the repo's real workflows: staged rollout, rollback, changelog — and flags deploy-only workflows that aren't real test gates.

🧩 Skills reference

Skill	Does	Status
`acef`	Front door. Routes intent, coordinates helpers, guides and teaches.	DRAFT
`acef-adapter`	Extract the per-repo project adapter (evidence-based, no embeddings).	DRAFT
`acef-router`	Classify a request into a route + the minimum questions.	DRAFT
`acef-specify`	Requirements/design/tasks — import if present, else lightweight.	DRAFT
`acef-test-bootstrap`	First approved golden test for a repo with none.	DRAFT
`acef-release-adapter`	Release/CD readiness from the repo's real config.	DRAFT
`codebase-map`	Auto-generate a code-grounded map of any repo (+ freshness stamp).	READY

Plus your project's own build and review helpers — ACEF calls them through the adapter, it doesn't replace them.

✅ Honest status

ACEF labels every capability so you always know what is real:

READY

Working skill or real artifact, used on real repos.

DRAFT

Written, research-informed definition — *not yet tested on a real project*.

MISSING

Not built or not found yet.

Today the orchestrator and helper skills are DRAFT; the methodology docs are written; `codebase-map` is READY. The next step is using the agent on a real case — after that, DRAFT skills can be promoted to READY.

📦 Setup

ACEF ships as AI-agent skills. The front door is the `acef` skill; it delegates to helpers and reads its own portable methodology docs, so it works in any repo.

```
skills/
  acef/                  # the orchestrator (front door) + references/ (portable m
  acef-adapter/          # project adapter extraction
  acef-router/           # route classification
  acef-specify/          # requirements / design / tasks (import or lightweight)
  acef-test-bootstrap/   # first golden test for a repo with no tests
  acef-release-adapter/  # release / CD readiness
  codebase-map/          # auto-generate a code-grounded map of any repo
```

Nothing in the skills is project-specific — every stack value comes from the adapter you extract per repo.

⚖️ Principles

- **Stack-agnostic.** The core knows no stack; every stack value comes from the adapter.
- **Code-grounded.** Copy a real golden neighbor; never invent a pattern the repo already has.
- **No ceremony.** Run only the steps a route needs. No dashboards or cadences unless asked.
- **Human approval on risk.** Auth, payment, migrations, secrets, shared APIs, releases, and multi-repo work need a human before execution.
- **Honesty.** Always label READY / DRAFT / MISSING. Never present a draft as proven.

THE PROCESS

SDLC & Frameworks

ACEF doesn't invent a new lifecycle. It overlays the normal SDLC, then *orchestrates* the frameworks you already use — BMAD v2, Kiro, spec-kit, GSD, OpenSpec — through your project's real adapter, running only the phases a given request needs.

🗺️ The shape

This guide starts at the first brief. Once the work is framed, ACEF routes into the SDLC phases below and only runs the parts that matter for that request.

1 Brief / Route → **2 Requirements** → **3 Design** → **4 Planning** → **5 Implement** → **6 Review/Test/CI** → **7 Release** → **8 Feedback**

ℹ️ Most requests touch only a few of these. A bug fix skips 2–4 entirely; a small feature skips heavy requirements/design. The router decides which phases run.

🔢 The 8 phases

1 · Brief & Route

entry

Take the brief, classify it into a route, and detect risk. Greenfield vs brownfield is a deterministic check, not a question.

Frameworks: ACEF router · BMAD scale levels (idea)

2 · Requirements

what to build

User-facing behavior + acceptance criteria, with traceable IDs. Imported if a tool already produced them.

Frameworks: BMAD PRD · Kiro requirements (EARS) · spec-kit FR/US/SC IDs

3 · Design

how to build it

Architecture, data flow, contracts, affected modules, risks.

Frameworks: BMAD architecture · Kiro design

4 · Planning

break it down

Story map → epics → user stories → tasks, each traced back to a requirement.

Frameworks: BMAD stories/tasks · storymap · OpenSpec deltas (brownfield)

5 · Implement

build

Copy the repo's golden neighbor; minimal, code-grounded change per task.

Frameworks: your project's own build skills, via the adapter

6 · Review / Test / CI

verify

Review against the repo's standard; bootstrap a test pattern if none exists; run the gate.

Frameworks: your test stack (detected) · review skills

7 · Release / CD

ship

Readiness checklist, staged rollout, rollback — from the repo's real workflows.

Frameworks: your CI/CD, surfaced via the release adapter

8 · Feedback

learn

What went wrong / what to change → update the rule or checklist. No metric ceremony unless asked.

Frameworks: lessons / retro

🅱️ BMAD v2 in detail

BMAD v2 is itself a full pipeline (Analysis → Planning → Solutioning → Implementation) with agent personas. ACEF does **not** run all of BMAD. It uses BMAD as the **large-feature planning engine** and imports its output — it never regenerates what BMAD already produced.

BMAD produces	ACEF maps it to
PRD (functional / non-functional requirements)	`requirements.md`
Architecture document	`design.md`
Epics / stories / tasks	`tasks.md`
Readiness report	Definition-of-Ready evidence

> ⚠️ BMAD also has its own brief/research/analysis phase, which *overlaps* ACEF's front. Rule of thumb: **generate once.** If BMAD already made the spec, ACEF imports + enriches it (risk, DoR, test cases, golden neighbor, traceability) — it does not run a second pipeline. On small or quick work, BMAD is skipped entirely.

🔌 Where each framework plugs in

ACEF orchestrates; it doesn't force you to use all of these. You use what your project already has, and ACEF imports the output through the adapter into one canonical format.

Framework	Plugs into	What ACEF takes
BMAD v2	Phases 2–4 (large feature)	PRD→requirements, architecture→design, stories→tasks; import, don't regenerate
Kiro	Phases 2–4 (spec format)	the 3-file `requirements/design/tasks` shape + EARS acceptance criteria
GitHub spec-kit	Phases 2–4 (traceability)	`FR-###` / `US#` / `SC-###` IDs + tasks tagged to stories
OpenSpec	Phase 4 (brownfield currency)	ADDED/MODIFIED/REMOVED deltas that keep living specs current
GSD	Phase 1 / onboarding	orientation + codebase mapping

The differentiator vs any single tool: ACEF binds all of them to *your* extracted adapter (per-repo, evidence-based) and runs them behind one agent — no ceremony, no double work.

🚦 Routes vs phases

A route is just a subset of phases. This is why "every job goes through all 8 phases" is false.

Route	Phases it runs
A · Small feature	1 → 5 → 6 (skip 2–4)
B · Large feature	1 → 2 → 3 → 4 → 5 → 6
C · Bug fix	1 → 5 → 6 (root-cause first)
D · Test cases	2 (flows→AC) → 6
E · Test automation	6 (bootstrap first)
F · Unit / integration	6

Route drawings

Same routes, but spelled out. Each card shows the shortest useful path for that job.

A · Small feature

Brief / route → **Golden neighbor** → **Implement** → **Review**

Use this when the change is small, local, and already matches an existing pattern.

B · Large feature

Brief / route → **Requirements** → **Design** → **Planning** → **Implement** → **Review**

Use this when scope is larger, a new contract appears, or multiple modules are affected.

C · Bug fix

Brief / route → **Reproduce** → **Root cause** → **Fix** → **Review**

Use this when the job is about restoring behavior, not adding a new feature shape.

D · Test cases

Brief / route → **Flows** → **Acceptance criteria** → **Test cases** → **Review**

Use this when the output is a test plan or structured cases before automation.

E · Test automation

Brief / route → **Detect stack** → **Bootstrap 1 test** → **Approve pattern** → **Expand**

Use this when the project needs the first real test pattern before broader automation can scale.

F · Unit / integration

Brief / route → **Detect stack** → **Unit / integration** → **Run gate**

Use this when the job is about test coverage in the project's actual framework and command set.

WHAT EACH ROLE CAN DO

ACEF for your role

Everyone talks to the same `acef` agent in plain language. You never pick routes, phases, or skills — you just describe what you want. This page shows **what each role can do** and which flows concern them.

🔭 One framework, different lenses

The engine is the same for everyone: it understands your project, routes the request, and runs only the steps that matter. What changes per role is the **lens** — which flows are yours.

DEVELOPER

builds and fixes — features, bugs, tests in code.

PM / ARCHITECT

frames, plans, and gates — scope, risk, go/kill.

TEST / QA

turns behavior into checks — flows, cases, automation.

👩‍💻 Developer

What you can do — describe the work in plain words; ACEF grounds it in your repo and reviews the result.

Add a small feature A

ACEF copies the pattern your repo already uses for that kind of screen/endpoint, wires your change, and reviews it.

Build a large feature B

It turns the work into requirements, design, and tasks, then builds each task — pausing for approval on risky parts.

Fix a bug C

Give the symptom; it reproduces, finds the root cause, and makes the smallest fix that matches your code's style.

Add unit / integration tests F

It writes tests in your project's real framework, around behavior — not implementation detail.

YOU SEE

the task · the real code to copy · files likely touched · the definition of done.

YOU DON'T DEAL WITH

specs, planning, or risk dashboards — that's handled around you.

DEVELOPER

The avatar disappears after editing a profile.

ACEF

Bug fix. I'll reproduce on a fresh profile, find the root cause, and fix it the way this repo handles image state — then add a regression test. Only after a refresh, or immediately?

🧭 Project manager / architect

What you can do — shape and control the work without touching code.

Frame a brief

Brief

Turn a raw idea into a clear brief: the problem, who it's for, success criteria, scope, and explicit non-goals.

Plan a feature

Planning

Break it into a story map and traceable tasks — every task links back to a requirement, so scope is auditable.

Get a risk read

Risk

See the risk class of any work and exactly which steps are guarded (auth, payment, migrations, multi-repo).

Gate the work

Go / Kill

Approve, hold, or kill at the gate. Guarded work cannot merge without your sign-off.

Track status

Status

See what's queued, building, in review, blocked, or done across the work in flight.

YOU SEE

scope & non-goals · the plan · risk · the go/kill gate · status.

YOU DON'T DEAL WITH

code or implementation detail — unless you ask for it.

PM

How big is team billing, and what's the risk?

ACEF

Large feature, high risk — it touches payment, adds a new contract, and spans the API and dashboard. It needs a brief, a plan, and your approval before the payment step merges. Want the brief drafted?

🧪 Test engineer / QA

What you can do — turn behavior into checks, in your project's real test setup.

Extract test cases — D

Turn product flows into acceptance criteria and structured test cases — ready to review or automate.

Write unit / integration tests — F

Tests written in your project's actual framework and command set, around real behavior.

Build test automation — E

Automate the priority user flows end-to-end, on the stack your repo already uses.

Bootstrap the first test — E / F

Zero tests in the repo? It sets up one approved example pattern that every later test follows.

Scale coverage by risk — All

Happy path for low-risk flows; negative, edge, permission, session, and failure cases for guarded ones.

A flow becomes structured cases like this:

Case	Precondition	Steps	Expected	Type
Login · happy	valid account	enter creds → submit	lands on dashboard	positive
Login · wrong password	valid account	wrong password → submit	inline error, no session	negative
Login · locked	locked account	submit	lockout message	edge
Login · session	logged in	refresh	stays authenticated	session

YOU SEE

flows · acceptance criteria · test cases · validation evidence.

YOU DON'T DEAL WITH

implementation internals — you work at the behavior level.

At a glance

Role	What you can do	Flows
Developer	add small/large features · fix bugs · write unit/integration tests	A · B · C · F
PM / architect	frame a brief · plan · risk read · gate (go/kill) · track status	brief · planning · gates
Test / QA	extract test cases · write unit tests · build automation · bootstrap first test · scale coverage	D · E · F

All three talk to the same agent — the lens (which flows are yours) is what changes.